EVEREST TEXTILE MILLS (ETMS)

System Architecture, Engineering Decisions & Evaluator Defense Dossier

A comprehensive technical guide breaking down all separation-of-concerns layers, database invariants, edge telemetry, and Q&A; defense scripts.

1. System Startup & Immediate Operational Testing

The software contains two functional interfaces that run concurrently without port collision against a unified modular backend:

Subsystem	Command Line to Execute	Port / URL	Purpose & Target Audience
Backend Core & Industrial Dashboard	.\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --port 8000 --reload	http://localhost:8000/	FastAPI REST API, SQLite/PostgreSQL Engine, and C
Swagger Open-API Documentation	Runs automatically with backend	http://localhost:8000/docs	Interactive browser testing of all 6 modular routers, req
LoomTrack React Frontend	cd react_frontend; npm start	http://localhost:3000/	React 19, Framer Motion 3D sliding doors, role dashbo
Invariant Test Suite	.\.venv\Scripts\python.exe -m pytest backend/tests/test_invariants.py	Cli / Terminal	Validates all 6 mathematical invariants: double-dispatch

2. Ground-Up Fundamentals: Separation of Concerns

When evaluators ask how the system processes data, explain that every mill workflow adheres strictly to a 6-tier unidirectional pipeline:

Layer	Project File / Location	Responsibility & Technical Explanation
1. Client State	frontend/js/store.js & react_frontend/src/api	Manages optimistic UI updates, local storage fallbacks, audio cues, and async network requests via
2. Router	backend/app/routers/*.py	Address book pairing HTTP verbs with URIs (e.g., POST /api/v1/yarn/inward). Contains zero busines
3. Middleware	backend/app/main.py & loomtrack_api.py	Intercepts requests pre-execution. Validates CORS headers, extracts Bearer JWT tokens, and check
4. Controller / Schemas	backend/app/schemas.py	Validates input payloads via Pydantic DTOs (e.g. InwardYarnDTO, DispatchScanDTO) before logic r
5. Domain Service	backend/app/costing.py & routers/quality.py	Executes textile business rules: ASTM D5430 defect grading, Activity-Based Costing (ABC), and saf
6. Persistence (DB)	backend/app/models.py & everest_manufacturing.db	Executes atomic ACID transactions using async SQLAlchemy, enforcing unique constraints and row

3. Mapping the 6 Everest Mill Scenes to the Architecture

Mill Scene	HTTP Route	Controller Handler	Role Gate	Invariant & Critical Edge Case
Scene 1: Gate Inward	POST /api/v1/inventory/inward-yarn	record_inward_yarn	store_manager	Rejects duplicate lot numbers with 409 Conflict. Tests
Scene 2: Floor Machines	GET /api/v1/machines POST /api/v1/simulate/tick	list_machines simulate_floor_tick	all_roles	Real-time pick counters and Amps. Flags machines idl
Scene 3: ASTM D5430 QC	POST /api/v1/quality/inspect	process_roll_qc	qc_inspector	Points/100m = (Points*100)/Meters. Rejects if >40. Au
Scene 4: Truck Dispatch	POST /api/v1/dispatch/scan-truck	scan_roll_to_truck	store_manager	Anti-Double Dispatch Guard: Blocks rolls already dispa
Scene 5: Buyer Tracking	GET /api/v1/orders/track/:order_no	track_order_for_customer	buyer, sales	Self-service milestone stepper (Spinning -> Weaving ->
Scene 6: Owner Financials	GET /api/v1/costing/order/:id GET /api/v1/finance/pnl-summary	compute_order_costing get_monthly_financial_pnl	owner	Activity-Based Costing: Raw yarn + machine energy ho

4. Evaluator Q&A; Defense Script (Defending Every Choice)

Use these precise, technically authoritative answers during your oral evaluation:

Evaluator: 'Why did you choose FastAPI over Express.js or Django?'

Answer: 'FastAPI provides native asynchronous I/O via Python ASGI (uvicorn/asyncio) alongside strict Pydantic type validation. For high-frequency industrial IoT telemetry—where thousands of machine sensor pulses and barcode scans hit the server every minute—FastAPI's async event loop handles concurrent connections with lower latency and higher throughput than synchronous Django, while offering native Open-API documentation that Express requires third-party packages to match.'

Evaluator: 'How does your system prevent a fabric roll from being loaded onto two trucks?'

Answer: 'We enforce this at the database transaction layer, not merely in frontend UI state. In routers/dispatch.py, the scan_roll_to_truck controller checks if a record already exists in the dispatch_items table. A unique constraint on dispatch_items.roll_id ensures that even under concurrent race conditions (such as two forklift operators scanning the same roll barcode at the exact same millisecond), the second transaction triggers an IntegrityError, rolls back atomically, and returns an explicit HTTP 409 Conflict. This invariant is proven by our pytest suite (test_parallel_concurrency_race_condition).'

Evaluator: 'Explain how your Quality Control grading works according to international standards.'

Answer: 'Our system implements the ASTM D5430 4-Point System standard. When an inspector inputs defect points and roll length, the formula Points_per_100m = (Total_Defect_Points * 100) / Length_Meters is executed. Rolls with <= 15 points receive Grade A, <= 25 Grade B, <= 40 Grade C, and > 40 are classified as REJECT. Crucially, uninspected (PENDING) and REJECTED rolls are hard-blocked by our dispatch controller from being loaded onto trucks.'

Evaluator: 'What is Activity-Based Costing (ABC) and why is it superior to traditional accounting?'

Answer: 'Traditional accounting spreads plant overhead evenly across products, hiding true costs. Our Activity-Based Costing engine (backend/app/costing.py) absorbs exact raw material weight multiplied by lot unit price, actual machine running hours multiplied by hourly machine overhead rates, and shift operator hours multiplied by labor wages for each specific production run. This gives the mill owner true gross margins per meter instead of arbitrary estimates.'

Evaluator: 'How does your frontend handle network drops on the factory floor?'

Answer: 'Our frontend architecture (frontend/js/store.js) features a resilient Dual-Mode state engine. If factory Wi-Fi drops, the client detects the offline status, handles UI updates optimistically, and enqueues operational transactions into localStorage (etms_offline_queue). Once connectivity restores, it syncs queued records against the backend ledger.'

Evaluator: 'Why do you have two frontends in this repository?'

Answer: 'The repository features a decoupled architecture serving two distinct user experiences: (1) The industrial Caterpillar VisionLink dashboard served at root / is engineered for floor supervisors and control-room monitors displaying live Modbus/RS-485 machine telemetry, accordions, and canvas sparklines. (2) The LoomTrack React 19 app is designed for executive role portals and customer transparency. Both frontends consume the exact same underlying SQLite/PostgreSQL operational database via unified API contracts.'